

CentraleSupelec

**Software Engineering Project 2026**

**MySupermarket Checkout System**

*Project Report*

**Students:**

Shriya Sengupta

Utkarsh Mangal

Supervisor: Prof. Paolo Ballarini

## **1. Introduction**

This report covers the design and implementation of the MySupermarket checkout system, a Java application built for the Software Engineering Project 2026 at CentraleSupélec. The system handles a typical supermarket point-of-sale workflow: item scanning, bill computation with discount plans and category-level pricing, bank card payment through a POS/TAS pipeline, home delivery with 2-hour time-slot booking and dynamic pricing, and live stock tracking with low-stock alerts.

The application runs through a command-line interface (CLUI) as required by Part 2 of the project brief. All requirements R1 to R10 are implemented. The project is run from Eclipse (right-click Main.java, Run As, Java Application)

## **2. Design Decisions**

This section explains how we approached the main requirements and what design choices we made.

### **2.1 Facade: SupermarketSystem**

We put all subsystem coordination inside one class called `SupermarketSystem`. This acts as a Facade so the CLI only needs to call methods on this one class and does not need to know how inventory, payment, or delivery work internally. It also made it easy to test individual subsystems in isolation.

### **2.2 Strategy pattern for discount plans (R5, R5b)**

Discount plans are defined by a `DiscountPlan` interface with three methods: `discountMultiplier(subtotal)`, `annualFee()`, and `deliveryFeeMultiplier()`. There are three implementations: `NormalPlan`, `PrimePlan`, and `PlatinumPlan`. A new plan can be added by writing a new class implementing the interface and registering it in `PlanRegistry`. No existing code changes. This satisfies R5b.

### **2.3 Category pricing at runtime (R6, R6b)**

Category discounts are stored in a `Map<String, Double>` inside `CategoryPricingPolicy`. When `setCategoryDiscount` is called with a new category name it just adds a map entry. There is no switch-case on category names, so R6b is satisfied without modifying existing classes.

### **2.4 Observer pattern for stock alerts (R9)**

After each successful payment, `InventoryManager` decrements stock. If any item falls below its threshold, it calls `onLowStock(Item)` on every registered observer. At startup two `ManagerAlertObserver` instances are registered (one for Manager, one for Supplier). Adding

a new alert target just means implementing `StockObserver` and calling `addObserver()`. No other code changes.

## 2.5 Role checks in the CLI

The CLI class calls `requireRole(class, cmd)` before running each command, using Java's `instanceof` on the currently logged-in user. This keeps all access control logic in one place.

## 2.6 Delivery service (R7, R8, R8b)

We split delivery into two methods in `DeliveryService`: `computeRawFee()` applies the weight and distance rules, and `computeFee()` multiplies the result by `plan.deliveryFeeMultiplier()`. Any new plan with a different delivery discount just needs to return a different multiplier. The delivery service never needs to change.

## 2.7 Dynamic delivery slots (R10)

The `DeliverySlot` class models a 2-hour delivery window with a type (NORMAL, PEAK, ECO\_FRIENDLY), a maximum vehicle capacity in kg, and a dynamic pricing multiplier (PEAK +30%, ECO\_FRIENDLY -20%). `DeliveryService` pre-loads seven daily slots (S1 to S7). The customer uses `showSlots <weightKg>` to see available slots and `selectSlot <slotId>` to book one. The `bookSlot()` method prevents overbooking by checking remaining capacity before committing. The final delivery fee is: raw fee x plan multiplier x slot multiplier.

## 2.8 Payment flow (R4, Sections 2.5 and 2.6 of the spec)

The `POSTerminal` handles the full sequence: card lookup, PIN check, opening a TAS connection, authorisation, and debit. A `simulatedOutcome` field lets tests force any result (SUCCESS, INSUFFICIENT\_FUNDS, PIN\_WRONG, AUTH\_DENIED) without touching bank state, which is used by the `simulatePayment` command.

## 2.9 Checkout as a snapshot

A `CheckoutSession` captures the customer, their plan, the active pricing policy, and any pending delivery address and slot ID at the moment `startCheckout` is called. Changes made by the manager during a checkout do not affect the current bill.

### 3. Design Patterns Used

| Pattern             | Where applied                                           | Why                                                                                         |
|---------------------|---------------------------------------------------------|---------------------------------------------------------------------------------------------|
| Facade              | SupermarketSystem                                       | Keeps the CLI simple. It only calls one class instead of reaching into all subsystems.      |
| Strategy            | DiscountPlan + NormalPlan / PrimePlan / PlatinumPlan    | Each plan is its own class. New plans can be added without changing existing code. (R5b)    |
| Observer            | InventoryManager + StockObserver + ManagerAlertObserver | Stock alerts go to any number of listeners without the inventory knowing who they are. (R9) |
| Registry            | PlanRegistry                                            | Central lookup for discount plans. New plans can be registered at runtime. (R5b)            |
| Open/Closed via Map | CategoryPricingPolicy                                   | New category names accepted at runtime with no code changes. (R6b)                          |

### 4. UML Diagrams

#### 4.1 Class Diagram

The diagram below shows the main classes and their relationships. <<interface>> labels mark interfaces. Arrows show inheritance. Aggregation is shown with 'o'.

```
<<interface>> DiscountPlan
+ getName(): String
+ discountMultiplier(subtotal: double): double
+ annualFee(): double
+ deliveryFeeMultiplier(): double
    ^           ^           ^
    |           |           |
NormalPlan    PrimePlan    PlatinumPlan
no fee,       50 EUR/yr,    200 EUR/yr,
no discount   20% if >=50 EUR  30% always,
              50% delivery    free delivery

PlanRegistry
- plans: Map<String, DiscountPlan>
+ register(plan) + get(name) + all()

<<abstract>> User
- firstName, surname, username, password
+ checkPassword() + getRole()
    ^           ^           ^
    |           |           |
Manager    Cashier    Customer
              - customerId: int
              - address: String
              - plan: DiscountPlan
              - bankCard: BankCard
              - pendingDeliveryAddress: String
              - pendingDeliverySlotId: String (R10)

SupermarketSystem (Facade)
- inventory: InventoryManager
- planRegistry: PlanRegistry
```

- pricingPolicy: CategoryPricingPolicy
- deliveryService: DeliveryService
- tas: TransactionAuthorisationSystem
- pos: POSTerminal
- users: Map<String, User>
- currentSession: CheckoutSession
- totalRevenue: double

#### DeliveryService

- slots: List<DeliverySlot> (R10)
- + computeRawFee() + computeFee() + bookSlot() + getAvailableSlots()

#### DeliverySlot (R10)

- slotId: String
- startTime / endTime: LocalTime
- type: SlotType { NORMAL, PEAK, ECO\_FRIENDLY }
- maxCapacityKg / currentLoadKg: double
- + hasCapacity() + book() + getPricingMultiplier()  
PEAK x1.30, ECO\_FRIENDLY x0.80, NORMAL x1.00

#### InventoryManager

- catalogue: Map<String, Item>
- observers: List<StockObserver>
- + addObserver() + decrementStock() + restock()  
o (notifies)

#### <<interface>> StockObserver

- + onLowStock(item: Item)

^

|

#### ManagerAlertObserver

- role: String ("Manager" or "Supplier")
- + onLowStock(item) -> prints LOW STOCK alert

#### POSTerminal

- tas: TAS
- simulatedOutcome
- + pay(card, pin, amount)  
: PaymentResult

#### TransactionAuthorisationSystem

- cards: Map<String, BankCard>
- connectionOpen: boolean
- + authorise(cardNum, amount): AuthResult

#### CheckoutSession

- customer: Customer
- cart: List<CartEntry>
- deliverySlotId: String (R10)
- pricingPolicy / deliveryService
- + addItem() + computeBill() + close()

#### Item

- name: String
- category: String
- unitPrice: double
- weightKg: double
- stock: int
- lowStockThreshold: int
- + decrementStock(qty) + restock(qty) + isBelowThreshold()

#### CartEntry

- item: Item
- quantity: int
- + rawSubtotal(): double
- + totalWeight(): double

#### BankCard

- cardNumber: String
- pin: String
- balance: double

```
+ checkPin(pin): boolean
+ debit(amount) + getBalance()
```

## 4.2 Use Case Diagram

This shows which user types can run which commands.

|                                          |                                                                                                                             |
|------------------------------------------|-----------------------------------------------------------------------------------------------------------------------------|
| <<system>> MySupermarket Checkout System |                                                                                                                             |
| Manager                                  | --> setup, registerCashier, registerCustomer<br>--> addItem, restock, setCategoryDiscount<br>--> showInventory, showRevenue |
| Cashier                                  | --> startCheckout, scanItem, computeBill<br>--> pay, simulatePayment                                                        |
| Customer                                 | --> subscribeToPlan, requestDelivery<br>--> showSlots, selectSlot (R10)                                                     |
| TAS/Bank                                 | --> authorise transaction (called by POSTerminal)                                                                           |
| All                                      | --> login, logout, help, runTest                                                                                            |

## 4.3 Sequence Diagram: Successful Checkout and Payment

The table below shows the happy-path flow: slot selection (R10), checkout, bill computation, and card payment through the POS/TAS pipeline.

| #                                                                                                         | From     | Dir | To       | Message / Label                 | Notes                          |
|-----------------------------------------------------------------------------------------------------------|----------|-----|----------|---------------------------------|--------------------------------|
| <b>Step 1 - Customer selects a delivery slot (R10)</b>                                                    |          |     |          |                                 |                                |
| 1                                                                                                         | Customer | →   | CLI      | showSlots 5.95                  | Customer logged in             |
| 2                                                                                                         | CLI      | →   | System   | showDeliverySlots(5.95)         |                                |
| 3                                                                                                         | System   | ←   | CLI      | lists S1..S7 with types         | S5 = 16:00-18:00, ECO_FRIENDLY |
| 4                                                                                                         | CLI      | ←   | Customer | display: available slots        |                                |
| 5                                                                                                         | Customer | →   | CLI      | selectSlot S5                   |                                |
| 6                                                                                                         | CLI      | →   | System   | selectDeliverySlot(alice, S5)   |                                |
| Slot S5 saved on Customer.pendingDeliverySlotId. No capacity booked yet (booking happens at computeBill). |          |     |          |                                 |                                |
| <b>Step 2 - Cashier opens checkout and scans items</b>                                                    |          |     |          |                                 |                                |
| 7                                                                                                         | Cashier  | →   | CLI      | startCheckout alice             | Cashier logged in              |
| 8                                                                                                         | CLI      | →   | System   | startCheckout(alice)            |                                |
| 9                                                                                                         | System   | →   | Session  | new CheckoutSession(alice, ...) |                                |
| Session snapshot: customer plan (prime), pricing policy, delivery address, slot ID (S5).                  |          |     |          |                                 |                                |
| 10                                                                                                        | Cashier  | →   | CLI      | scanItem milk 2                 |                                |

|                                                                                                                                                                                                          |           |   |           |                                                                                                   |                               |
|----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|-----------|---|-----------|---------------------------------------------------------------------------------------------------|-------------------------------|
| 11                                                                                                                                                                                                       | CLI       | → | Session   | addItem(milk, 2)                                                                                  | <i>Added to cart</i>          |
| 12                                                                                                                                                                                                       | Cashier   | → | CLI       | scanItem steak 1 (etc.)                                                                           |                               |
| 13                                                                                                                                                                                                       | CLI       | → | Session   | addItem(steak, 1) (etc.)                                                                          | <i>Repeated for all items</i> |
| <b>Step 3 - Bill computation</b>                                                                                                                                                                         |           |   |           |                                                                                                   |                               |
| 14                                                                                                                                                                                                       | Cashier   | → | CLI       | computeBill                                                                                       |                               |
| 15                                                                                                                                                                                                       | CLI       | → | Session   | computeBill()                                                                                     |                               |
| <i>Session applies: dairy -5%, fruit -10%, then prime plan (subtotal 23.24 &lt; 50 EUR, plan does not fire), then delivery: 15.00 x 0.5 (prime) x 0.80 (ECO_FRIENDLY) = 6.00 EUR. Total = 29.24 EUR.</i> |           |   |           |                                                                                                   |                               |
| 16                                                                                                                                                                                                       | Session   | ← | CLI       | totalDue = 30.74 EUR                                                                              | <i>Printed to console</i>     |
| <b>Step 4 - Payment (POS and TAS)</b>                                                                                                                                                                    |           |   |           |                                                                                                   |                               |
| 17                                                                                                                                                                                                       | Cashier   | → | CLI       | pay 4242424242424242 1234                                                                         |                               |
| 18                                                                                                                                                                                                       | CLI       | → | System    | pay(cardNum, pin)                                                                                 |                               |
| 19                                                                                                                                                                                                       | System    | → | POS       | pos.pay(card, pin, 30.74)                                                                         |                               |
| 20                                                                                                                                                                                                       | POS       | → | TAS       | openConnection(terminalId)                                                                        | <i>Secure channel opened</i>  |
| 21                                                                                                                                                                                                       | POS       | → | TAS       | authorise(cardNum, 30.74)                                                                         |                               |
| <i>TAS: verifies PIN matches card chip, checks balance &gt;= 30.74, debits account, credits shop.</i>                                                                                                    |           |   |           |                                                                                                   |                               |
| 22                                                                                                                                                                                                       | TAS       | ← | POS       | AuthResult.AUTHORISED                                                                             |                               |
| 23                                                                                                                                                                                                       | POS       | → | TAS       | closeConnection()                                                                                 |                               |
| 24                                                                                                                                                                                                       | POS       | ← | System    | PaymentResult.SUCCESS                                                                             |                               |
| <b>Step 5 - Post-payment: inventory update and Observer alerts</b>                                                                                                                                       |           |   |           |                                                                                                   |                               |
| 25                                                                                                                                                                                                       | System    | → | Inventory | decrementStock(cart)                                                                              |                               |
| <i>InventoryManager decrements each sold item. If stock &lt; lowStockThreshold -&gt; onLowStock() called on all registered observers.</i>                                                                |           |   |           |                                                                                                   |                               |
| 26                                                                                                                                                                                                       | Inventory | ← | System    | [ALERT -> Manager] LOW STOCK: 'steak' (category: meat) - only 4 unit(s) remaining (threshold: 5). | <i>Observer fires</i>         |
| 27                                                                                                                                                                                                       | System    | ← | CLI       | AUTHORISED - receipt printed                                                                      |                               |

## 5. Requirements Coverage

| Req. | Description                                                     | Status | Key classes                        |
|------|-----------------------------------------------------------------|--------|------------------------------------|
| R1   | System handles customer purchases                               | Done   | SupermarketSystem, CheckoutSession |
| R2   | A purchase is a collection of bought items                      | Done   | CartEntry, CheckoutSession         |
| R2b  | Item categories: fruit-and-vegetables, dairy, meat (and custom) | Done   | Item, CategoryPricingPolicy        |

|     |                                                                                                    |      |                                                                 |
|-----|----------------------------------------------------------------------------------------------------|------|-----------------------------------------------------------------|
| R3  | Items have a price; price can depend on quantity                                                   | Done | Item, CartEntry.rawSubtotal()                                   |
| R4  | Payment by debit or credit card                                                                    | Done | POSTerminal, BankCard, TAS                                      |
| R5  | Prime: 20% off when subtotal $\geq$ 50 EUR. Platinum: 30% always.                                  | Done | PrimePlan, PlatinumPlan                                         |
| R5b | New discount plans can be added without changing existing code                                     | Done | DiscountPlan interface, PlanRegistry                            |
| R6  | Category-level pricing policies (e.g. -10% on fruit-and-vegetables)                                | Done | CategoryPricingPolicy                                           |
| R6b | New categories and policies can be added at runtime                                                | Done | CategoryPricingPolicy (Map-based)                               |
| R7  | Customer can request home delivery                                                                 | Done | Customer, CheckoutSession                                       |
| R8  | Delivery fee based on total weight and distance. Orders over 50 kg refused.                        | Done | DeliveryService                                                 |
| R8b | Delivery discounts per plan: prime pays 50%, platinum pays nothing                                 | Done | DiscountPlan.deliveryFeeMultiplier()                            |
| R9  | Live stock tracking with Observer alerts when perishables go below threshold                       | Done | InventoryManager, StockObserver, ManagerAlertObserver           |
| R10 | 2-hour delivery time slots, overbooking prevention, dynamic pricing (PEAK +30%, ECO_FRIENDLY -20%) | Done | DeliverySlot, DeliveryService.bookSlot(), showSlots, selectSlot |

## **6. Design Analysis**

### **6.1 What works well**

- Both discount plans and category pricing can be extended without changing existing code, which directly satisfies R5b and R6b.
- The simulatedOutcome field in POSTerminal lets any payment outcome be tested without touching bank state, making it simple to write reproducible tests for failure cases.
- The CLI only calls SupermarketSystem, so it stays clean and does not need to know how the Observer chain, TAS, or delivery slots work.
- Adding a new stock observer only requires implementing StockObserver and calling addObserver(). Nothing else changes.
- Delivery slot booking prevents overbooking by checking remaining vehicle capacity before committing, and the dynamic pricing multiplier (PEAK/ECO\_FRIENDLY/NORMAL) is fully integrated into the bill computation.
- Pricing policies and slot selections are captured at checkout start, so changes made by the manager mid-session do not affect the current bill.



## 6.2 Current limitations

- Everything is stored in memory. Restarting the application loses all data. Adding a file or database layer would fix this.
- Only one checkout session can be active at a time because SupermarketSystem holds a single currentSession field. Supporting multiple tills would need a map from cashier to session.
- The distance estimate in DeliveryService uses a simple character-sum formula rather than a real geocoding API.
- Passwords are stored as plain text. A real system would hash them.

## 7. Test Scenarios

### 7.1 Configuration file: my\_supermarket.ini

This file loads automatically at startup. It sets up:

- Logs in as the default CEO manager.
- Runs setup, which loads the default catalogue (apple, banana, tomato, milk, yogurt, cheese, steak, chicken), two pre-registered test cards (4242424242424242 and 1111222233334444), a default cashier and default customer.
- Sets category discounts: fruit-and-vegetables -10%, dairy -5%.
- Adds extra items: orange, carrot (fruit-and-vegetables), butter (dairy), lamb (meat).
- Registers cashiers bob (bobpwd) and carol (carolpwd).
- Registers customers alice and pierre, each with an auto-generated bank card with 1000 EUR balance.
- Logs out.

### 7.2 Test scenario 1 (testScenario1.txt) - end-to-end happy and sad path

This covers all mandatory requirements. Run it with:

```
runTest testScenario1.txt
```

Steps:

- Manager logs in, runs setup, then adds items including steak with initial stock 5 (threshold also 5).
- -10% category discount applied to fruit-and-vegetables.
- Alice logs in, subscribes to prime (50 EUR annual fee deducted immediately), and requests home delivery to '12 rue de la Paix'.
- Cashier bob opens checkout for alice. Items scanned: 2x milk, 3x yogurt, 1x steak, 10x apple, 4x tomato.
- computeBill: no discount on dairy (milk 2.40, yogurt 2.40), no discount on steak (12.50), -10% on fruit-and-vegetables (apple 2.70, tomato 3.24). Subtotal = 23.24 EUR. Prime plan does not fire (below 50 EUR). Delivery: weight 5.95 kg, distance 12 km, flat fee 15.00 EUR  $\times$  0.5 (prime) = 7.50 EUR. Total due: 30.74 EUR.
- First payment: simulatePayment INSUFFICIENT\_FUNDS, pay  $\rightarrow$  refused.

- Second payment: simulatePayment SUCCESS, pay → authorised. Stock decremented: steak goes to 4, below threshold 5. Observer fires alerts to Manager and Supplier.
- Second checkout: 4x steak = 50.00 EUR, prime fires (20% off) → total 40.00 EUR. Payment SUCCESS.
- Manager: showInventory shows steak with [LOW STOCK] marker, showRevenue shows 70.74 EUR.

### 7.3 Expected outputs from test scenario 1

| What to check                               | Expected output                                                                                  |
|---------------------------------------------|--------------------------------------------------------------------------------------------------|
| Delivery fee line in bill (checkout 1)      | Delivery fee charged: 7.50 EUR                                                                   |
| Bill total for checkout 1                   | TOTAL DUE: 30.74 EUR                                                                             |
| Failed payment attempt                      | [POS] Payment REFUSED: insufficient funds.                                                       |
| Observer alert after checkout 1 (steak = 4) | [ALERT → Manager] LOW STOCK: 'steak' (category: meat) - only 4 unit(s) remaining (threshold: 5). |
| Observer alert after checkout 2 (steak = 0) | [ALERT → Manager] LOW STOCK: 'steak' (category: meat) - only 0 unit(s) remaining (threshold: 5). |
| Bill total for checkout 2 (prime fires)     | TOTAL DUE: 40.00 EUR                                                                             |
| steak in showInventory                      | steak ... 0 [LOW STOCK]                                                                          |
| showRevenue at end                          | Total revenue: 70.74 EUR                                                                         |

### 7.4 Test scenario 2 (R10): dynamic slot pricing

This short scenario tests the slot booking and dynamic pricing from R10. It can be added to testScenario1.txt or run as a separate file.

```
# After the ini has loaded (alice registered, prime plan active)
login alice alicepwd
requestDelivery "12 rue de la Paix"
showSlots 5.95
# Expected output: lists S1-S7 with their type and remaining capacity
# S5 is 16:00-18:00, ECO_FRIENDLY, 200 kg remaining
selectSlot S5
# Expected: [INFO] Delivery slot S5 (16:00-18:00, ECO_FRIENDLY) selected for Alice Martin.
logout

login bob bobpwd
startCheckout alice
scanItem milk 2
scanItem steak 1
computeBill
```

```
# Expected: raw fee = 15.00, prime multiplier 0.5 -> 7.50, ECO_FRIENDLY 0.80 ->
6.00
# Item subtotal 2*1.20*0.95 + 12.50 = 14.78 (below 50 EUR, prime does not fire)
# TOTAL DUE: 14.78 + 6.00 = 20.78 EUR
simulatePayment SUCCESS
pay 4242424242424242 1234
logout
```

Key thing to verify: *the slot S5 ECO\_FRIENDLY multiplier (x0.80) reduces the delivery fee below the normal prime rate. Choosing slot S2 (PEAK, x1.30) would give  $7.50 \times 1.30 = 9.75$  EUR instead.*

## **8. How to Run and Test the Implementation**

### **8.1 Running the application**

The primary way to run the project is from Eclipse:

- Open the project in Eclipse (File → Import → Existing Projects into Workspace).
- Right-click on src/cli/Main.java in the Package Explorer.
- Click Run As → Java Application.
- The CLUI starts in the Console tab. The my\_supermarket.ini file loads automatically if it is in the project root folder.

To run a test scenario from inside the application:

```
runTest testScenario1.txt
```

### **8.2 Running JUnit tests**

JUnit 5 tests are provided for all non-trivial methods. To run them in Eclipse: right-click the project → Run As → JUnit Test. All test classes are in the same package as the class they test. Test classes:

- ItemTest: stock decrement, restock, threshold flag.
- DiscountPlanTest: PrimePlan threshold (below and above 50 EUR), PlatinumPlan unconditional discount, annual fees.
- CategoryPricingPolicyTest: multiplier calculation, unknown category returns 1.0.
- DeliveryServiceTest: 50 kg refusal, flat fee, heavy and distance surcharges, plan multiplier, slot dynamic pricing.
- InventoryManagerTest: catalogue operations, Observer notification on low stock.
- BankTest: card not found, wrong PIN, insufficient funds, successful payment.
- SupermarketSystemTest: full checkout cycle, subscribe-to-plan fee charge, scanItem stock check, slot booking.
- CLITest: command routing, role enforcement, syntax error handling.

## **9. Workload Distribution**

The table below shows who had primary responsibility for each component. Both students contributed to all phases.

| <b>Class / Task</b>                                               | <b>Design</b> | <b>Code</b> | <b>UML Diagrams</b> | <b>JUnit Tests</b> | <b>Lines ~</b> |
|-------------------------------------------------------------------|---------------|-------------|---------------------|--------------------|----------------|
| SupermarketSystem (Facade)                                        | Utkarsh       | Utkarsh     | Both                | Shriya             | 200            |
| DiscountPlan + Plans + PlanRegistry                               | Both          | Shriya      | -                   | Utkarsh            | 130            |
| CategoryPricingPolicy                                             | Shriya        | Shriya      | -                   | Utkarsh            | 50             |
| InventoryManager + Observer classes                               | Utkarsh       | Utkarsh     | Both                | Shriya             | 100            |
| DeliveryService + DeliverySlot (R10)                              | Shriya        | Shriya      | Utkarsh             | Both               | 150            |
| POSTerminal + TAS                                                 | Utkarsh       | Utkarsh     | Both                | Shriya             | 120            |
| CLI + Main                                                        | Both          | Both        | -                   | Both               | 320            |
| Model classes (Item, CartEntry, BankCard)                         | Shriya        | Shriya      | -                   | Utkarsh            | 130            |
| Store classes (User, Customer, Cashier, Manager, CheckoutSession) | Utkarsh       | Utkarsh     | -                   | Shriya             | 220            |
| Report                                                            | Both          | -           | Both                | -                  | -              |
| Test files (.ini, testScenario1.txt)                              | Both          | Both        | -                   | -                  | 80             |

Total source lines excluding tests: approximately 1,600. Test source lines: approximately 650.

## **10. Conclusion**

The MySupermarket system covers all requirements from R1 to R10. The design uses a Facade to keep the CLI simple, Strategy for discount plans, Observer for stock alerts, and a runtime-extensible Map for category pricing. R10 is fully implemented: DeliverySlot models 2-hour windows with PEAK, NORMAL, and ECO\_FRIENDLY types, bookSlot() enforces vehicle capacity limits, and dynamic pricing is applied on top of the plan discount when the bill is computed.

The system was tested through the CLUI using testScenario1.txt and interactively. JUnit tests cover all the main logic. The project runs from Eclipse with a single right-click.